

Chapter 6

Encapsulation

Object-oriented TypeScript, test first

Bank Account

Aoife Byrne · account IE-0001

Balance: **€20.00**

€ 50

Deposit

Withdraw

Not enough money: the balance is €20.00

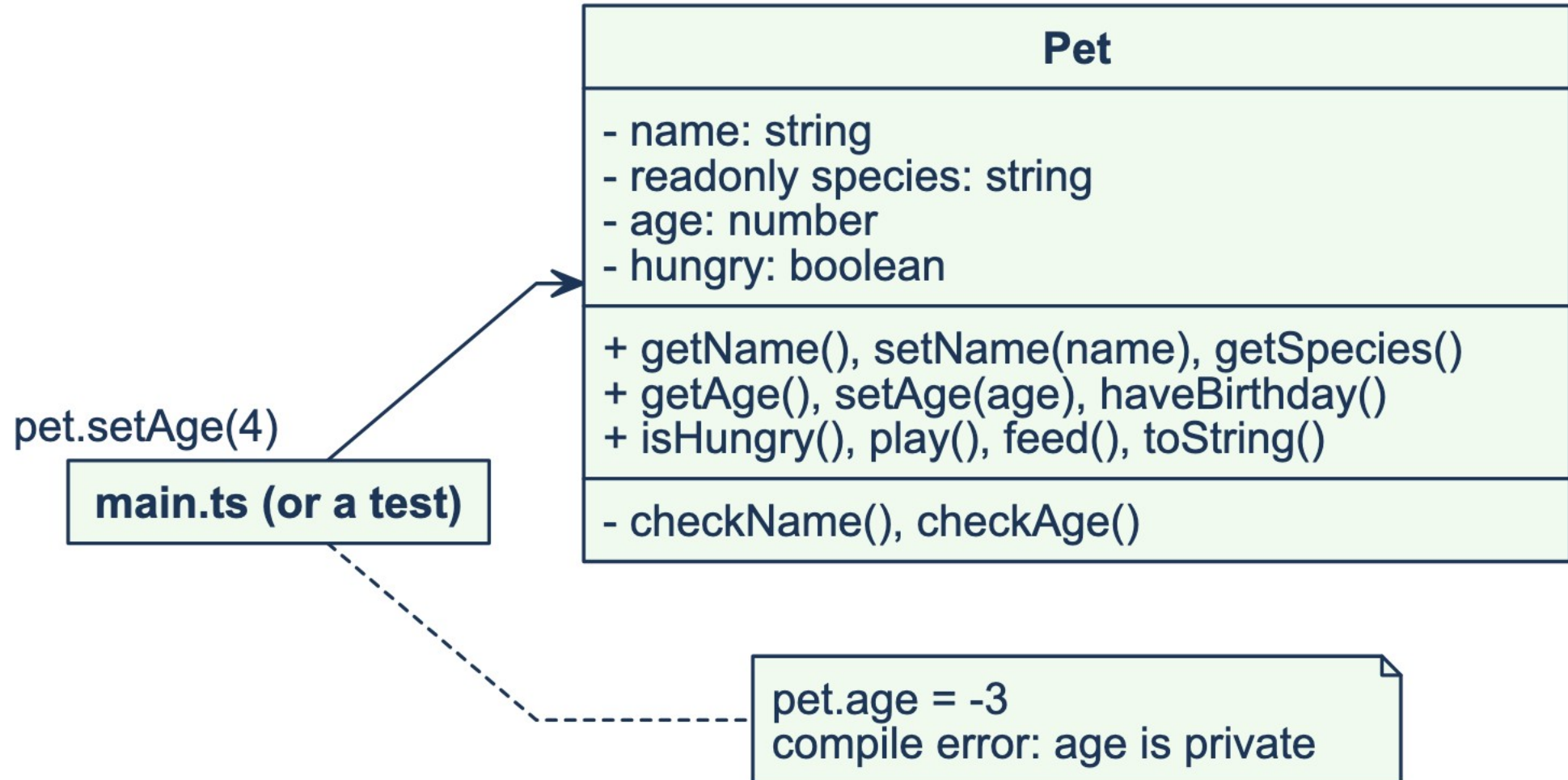
Today

- why an object hides its data
- `private`, `#private` and `readonly`
- getters and setters – methods and accessors
- setters that **refuse**, test first
- invariants: rules that are always true
- `try / catch` on the page

Why hide data?

- if any code can set `age = -3`, every bug can too
- hide the data; let methods change it - and **check** it
- the class can then *promise* things: "age is never negative"
- Chapter 2's `Counter` already did this: `count` was private

The outside and the inside



Pet - the Java way

```
export class Pet {  
  private name: string;  
  private readonly species: string;  
  private age: number;  
  private hungry: boolean = false;  
  
  public getName(): string {  
    return this.name;  
  }  
  
  public setName(name: string): void {  
    this.name = this.checkName(name);  
  }  
}
```

The compiler guards **private**

```
const pet = new Pet("Rex", "dog", 3);  
pet.age = -3;
```

TS2341 [ERROR]: Property 'age' is private and only accessible within class 'Pet'.

- private **methods** too: **checkName**, **checkAge**
- no visibility written = **public** (Java: package-private)

`isX()` and actions

- boolean getter: `isHungry()` , as in Java
- no `setHungry(true)` - instead `play()` and `feed()`
- methods that say **what happens**, not which field changes
- a setter for every field is a habit worth questioning

readonly - Java's final

```
public adopt(): void {  
    this.species = "cat";  
}
```

TS2540 [ERROR]: Cannot assign to 'species' because it is a read-only property.

- set in the declaration or the constructor - then never again

Red: a setter that says no

```
Deno.test("setAge refuses a negative age", () => {  
  const pet = new Pet("Rex", "dog", 3);  
  
  assertThrows(() => pet.setAge(-1), Error,  
    "Age must be a whole number");  
  assertEquals(pet.getAge(), 3);  
});
```

error: AssertionError: Expected function to throw.

Green

```
public setAge(age: number): void {  
  if (age < 0) {  
    throw new Error(  
      `Age must be a whole number, 0 or more (not ${age})`  
    );  
  }  
  this.age = age;  
}
```

- check **before** change: `throw` leaves at once
- next red: `setAge(2.5)` - add `!Number.isInteger(age)`

Red again: the constructor

- `new Pet("Rex", "dog", -1)` still works
- Expected function to throw.
- copy the `if` ? Then the rule is written twice...

Refactor: one copy of the rule

```
public setAge(age: number): void {  
  this.age = this.checkAge(age);  
}  
  
private checkAge(age: number): number {  
  if (!Number.isInteger(age) || age < 0) {  
    throw new Error(`Age must be ... (not ${age})`);  
  }  
  return age;  
}
```

- constructor: `this.age = this.checkAge(age);`

Why not `this.setAge(age)` ?

```
TS2564 [ERROR]: Property 'name' has no initializer  
and is not definitely assigned in the constructor.
```

- TypeScript does not look inside called methods
- and parameter properties skip checks - only for fields with no rules

Catching it on the page

```
try {  
    pet.setName(nameBox.value);  
    showMessage("");  
} catch (error) {  
    showMessage(error instanceof Error  
        ? error.message : String(error));  
}
```

- one `catch ; error` is `unknown` (anything can be thrown)

Who does what

Pet

(PET) Rex is a dog, and is 4 years old.

Rex is hungry!

Play Feed Birthday

Rename

A pet needs a name

- `Pet` decides what is allowed, and says why not
- `main.ts` decides how to show it

How private is `private`?

```
pet["age"] = -3;  
console.log(pet.toString());
```

(PET) Rex is a dog, and is -3 years old.

- no type error, no lint warning
- the build removes `private`: `dist/app.js` has plain `age`;

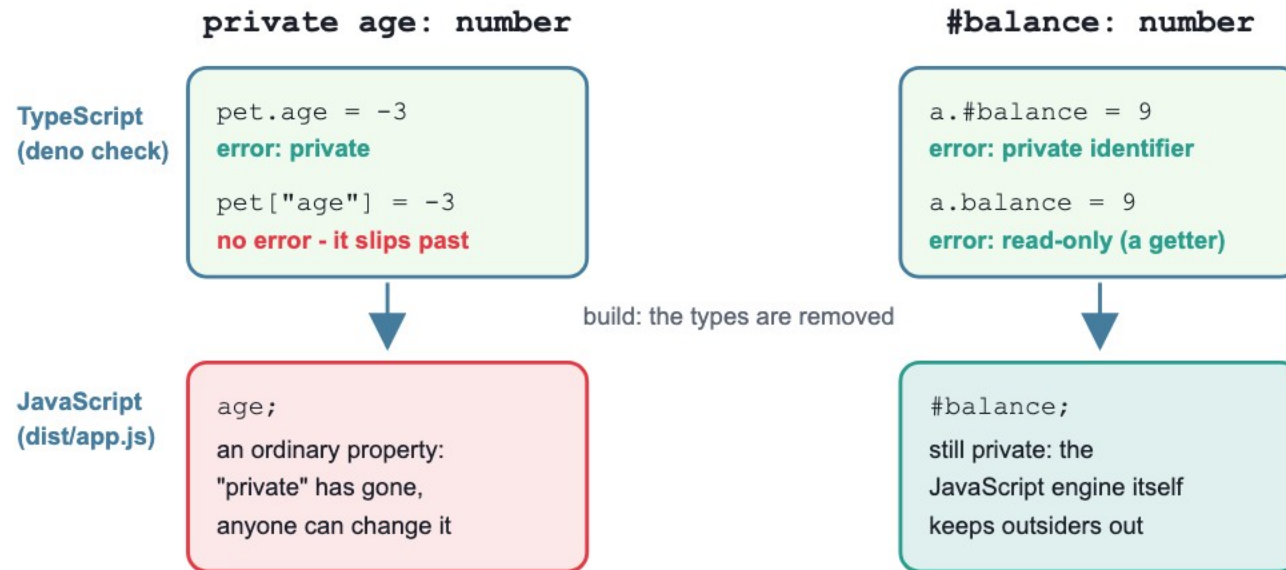
#private - real privacy

```
export class BankAccount {  
  #balance: number = 0;  
  
  constructor(public readonly owner: string,  
    public readonly accountNumber: string) {}  
}
```

TS18013 [ERROR]: Property '#balance' is not accessible outside class 'BankAccount' because it has a private identifier.

private vs

private age vs #balance



A getter with no setter

```
public get balance(): number {  
    return this.#balance;  
}
```

- read like a field: `account.balance` - no brackets
- `account.balance = 5` is a compile error: read-only
- `#balance` + `balance` : no naming clash
- money in whole **cents** - decimals are not exact

The invariant

the balance is a whole number of cents,
and never below zero

```
public withdraw(cents: number): void {  
    this.#checkAmount(cents);  
    if (cents > this.#balance) {  
        throw new Error(`Not enough money: ...`);  
    }  
    this.#balance -= cents;  
}
```

Change first, check after?

```
this.#balance -= cents;  
if (this.#balance < 0) { throw new Error(...); }
```

```
-   -1000  
+   2000
```

- it throws - **after** the damage
- a refusal must leave the object exactly as it was

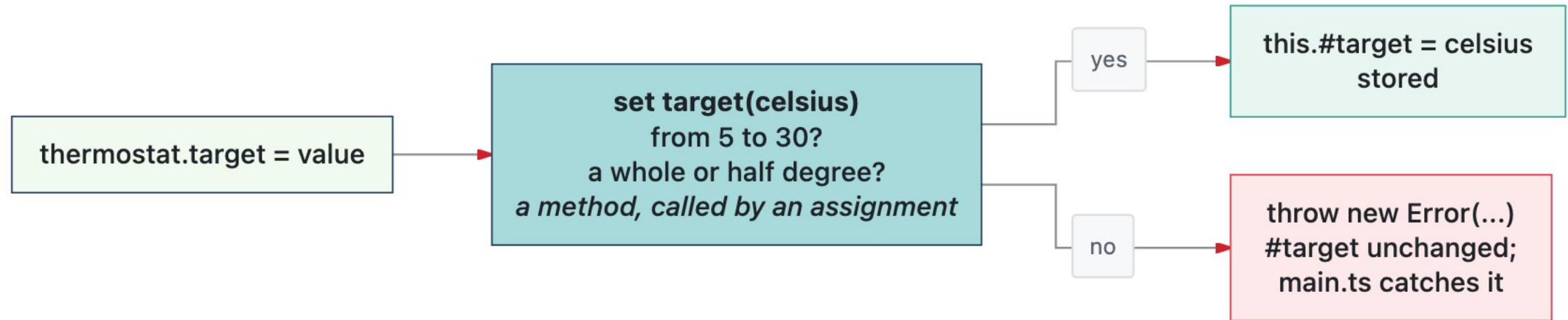
Many bad values: `t.step`

```
for (const amount of [0, -500, 12.5, NaN]) {  
  await t.step(`deposit(${amount}) is refused`, () => {  
    assertThrows(() => account.deposit(amount),  
      Error, "An amount must be");  
  });  
  // ... and the same for withdraw  
}  
assertEquals(account.balance, 2000);
```

The thermostat: **get** and **set**

```
public get target(): number {  
    return this.#target;  
}  
  
public set target(celsius: number) {  
    if (celsius < MIN_TARGET || celsius > MAX_TARGET) {  
        throw new Error(`The target must be ...`);  
    }  
    // ... half degrees only  
    this.#target = celsius;  
}
```

An assignment runs a method



```
assertThrows(() => {  
  thermostat.target = 30.5;  
}, Error, "The target must be from 5 to 30 °C");
```

Inside the class

```
public up(): void {  
    this.target = Math.min(this.#target + STEP,  
        MAX_TARGET);  
}  
  
public get heating(): boolean {  
    return this.room < this.#target;  
}
```

- `up()` goes **through the setter**: rules in one place
- `heating` is computed - never out of date

Not everything needs hiding

```
constructor(public room: number) {}
```

- no rules, so a public field is fine
- later it can become `get room()` / `set room(...)`
- `thermostat.room = 22` keeps working - no caller changes
- Java cannot do that, hence getters "just in case"

Accessors or methods?

- `get` : cheap, no parameters, no side effects
- `set` : setting is all that happens, with a check
- method: it **does** something - `deposit(500)` , `feed()`
- `getX()` / `setX()` are never wrong - accessors are neater
- either way: check first, refuse with an `Error`

Java and TypeScript

Java	TypeScript
<code>private</code> (also at run time)	<code>private</code> (compiler only)
-	<code>#field</code> (at run time too)
<code>final</code> field	<code>readonly</code>
<code>getAge()</code> / <code>setAge(4)</code>	same, or <code>pet.age</code> / <code>pet.age = 4</code>
<code>catch (IllegalArgumentException e)</code>	<code>catch (error) + instanceof Error</code>

Challenges

1. Tidy names - trim, at most 20 characters
2. Fahrenheit - a computed read-only property
3. An overdraft limit
4. Transfers - all or nothing
5. Pet, the TypeScript way - accessors
6. Holiday mode for the thermostat

Next: Chapter 7 - Model and view